

Fallstudie Textanalyse

MQM FS 2026 – Eigene Fallstudie zu Textdaten

Sarah Inderbitzin, Andri Schneider

01.06.2026

Inhaltsverzeichnis

1 Ausgangslage und Story	1
2 Daten	2
3 Vorverarbeitung	2
4 Explorative Analyse	2
4.1 Worthäufigkeiten	2
5 Methode 1 – Bag-of-Words & TF-IDF (Baseline, W11)	3
5.1 Distinktive Begriffe je Klasse (TF-IDF)	3
6 Methode 2 – Klassifikation auf Basis von Text (Naive Bayes, W11 S.44)	3
7 Methode 3 – Sentimentanalyse mit Wortlisten (W12 S.22-24)	4
8 Optionale Methoden (Skelette – aktivieren bei Bedarf)	4
word2vec (W12, S.8-14)	4
SBERT / Satz-Embeddings (W12, S.18; Python via reticulate)	5
Topic Modeling – LDA (W13)	5
LLM-Klassifikation (W12, S.19)	5
9 Interpretation der wichtigsten Ergebnisse	5
10 Kritische Diskussion	5
11 Fazit	6
12 Anhang: Code	7

1 Ausgangslage und Story

Vorgaben (Folien W12, S.29): Die Analyse soll *interpretierbar* sein, die Resultate *kritisch diskutiert* und in eine *kleine Story* verpackt werden. Als Vorbild dient die Vorlesungs-Fallstudie zu den TripAdvisor-Reviews des Restaurants *Clouds* (Rating, Reviewtext, Sprache, Übersetzung).

Beschreibe in 1–2 Absätzen:

- **Datenquelle & Kontext:** Woher kommen die Texte (Reviews, Forenbeiträge, Support-Tickets, ...)? Wer hat sie wann geschrieben? Sprache(n)?

- **Leitfrage:** z.B. “Was unterscheidet positive von negativen Bewertungen?”, “Welche Themen tauchen am häufigsten auf?” oder “Können wir aus dem Text automatisch positiv/negativ ableiten?” (vgl. Folien W12, S.3).
- **Erwartung / Hypothese:** Was vermutet ihr – und woran testet ihr es?

2 Daten

Anforderung (W12, S.28): Korpus mind. 200–300 Texte.

Tabelle 1: Eckdaten des Korpus

n_texte	anteil_positiv	woerter_im_mittel
20	0.5	7.25

Kurz beschreiben: Umfang, Klassenbalance, Textlänge, Auffälligkeiten.

3 Vorverarbeitung

Pipeline (W11, S.14): Rohtext -> Tokenisierung -> Normalisierung -> Feature-Matrix -> Modell. Preprocessing ist Teil der Modellierungsentscheidung (W11, S.38) – jede Entscheidung kurz begründen.

Begründe: Stoppwortliste passend? Stemming sinnvoll oder verzerrend (z.B. unregelmässige Verben, W11 S.31)? Seltene Wörter / Zahlen / Produktnamen behalten oder entfernen (W11, S.36/38)?

4 Explorative Analyse

4.1 Worthäufigkeiten

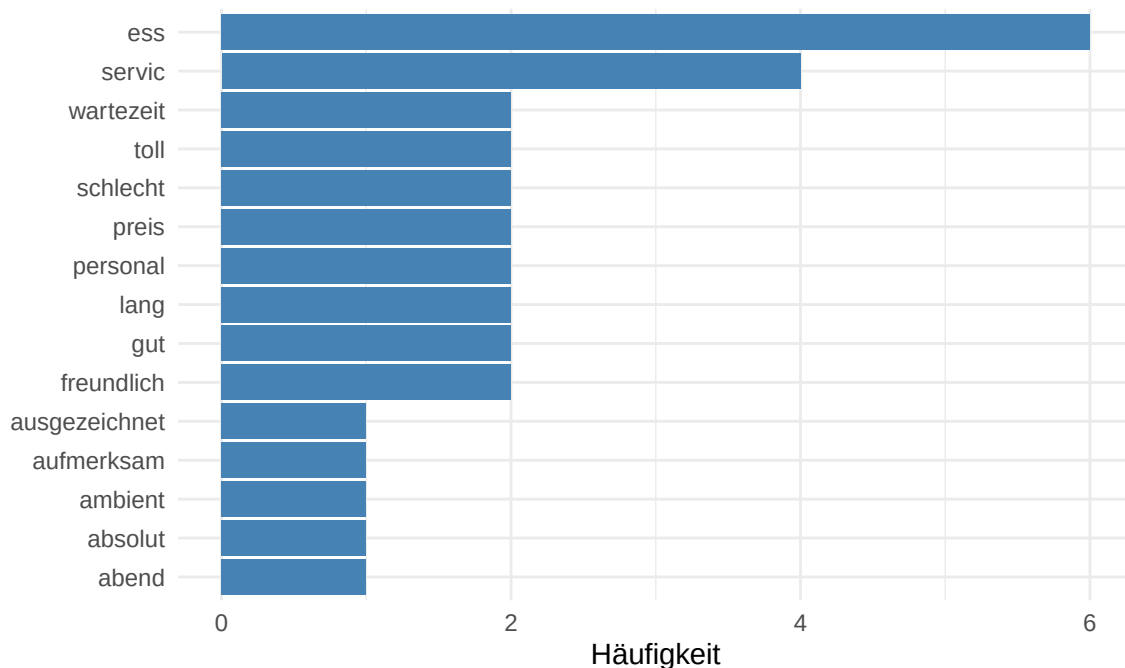


Abbildung 1: Häufigste Stems im Korpus

5 Methode 1 – Bag-of-Words & TF-IDF (Baseline, W11)

5.1 Distinktive Begriffe je Klasse (TF-IDF)

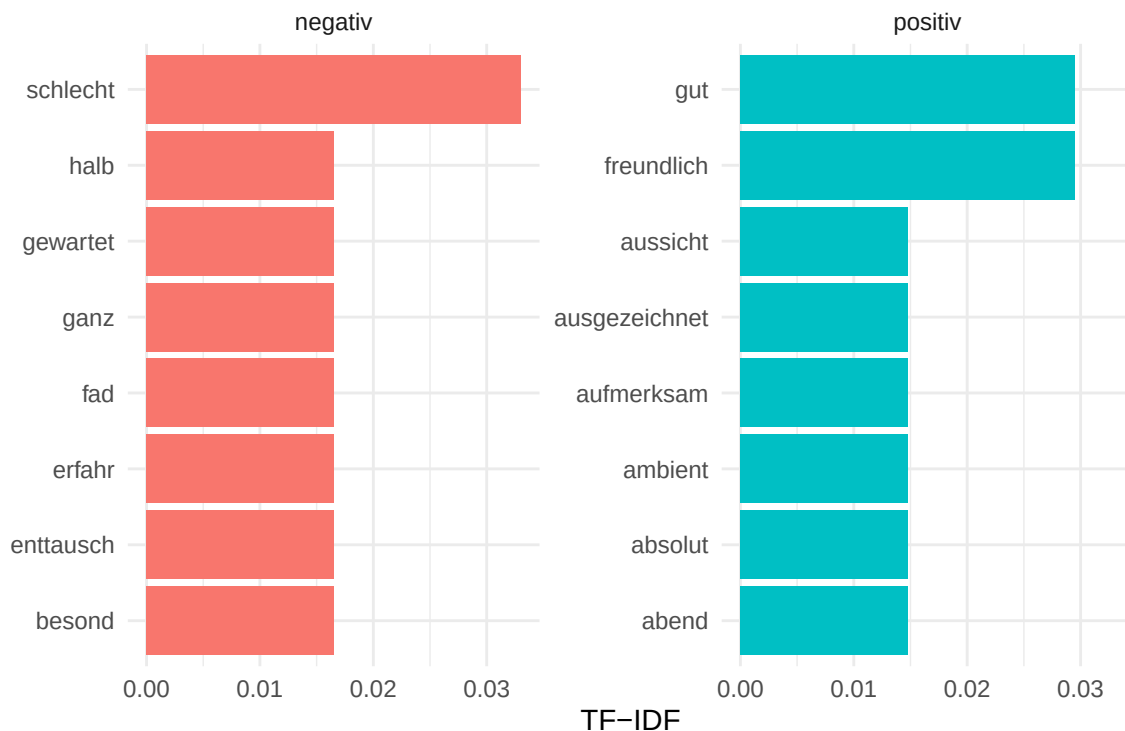


Abbildung 2: Top TF-IDF Stems je Klasse

Interpretation: Was sagen die distinktiven Wörter über die Gruppen aus?

6 Methode 2 – Klassifikation auf Basis von Text (Naive Bayes, W11 S.44)

Methodisch zentral (W11, S.45): Die Wortliste ist Teil des Trainings. Wortliste und Rare-Word-Filter NUR auf den Trainingsdaten bestimmen und unverändert auf die Testdaten anwenden – sonst fließt Testinformation ein.

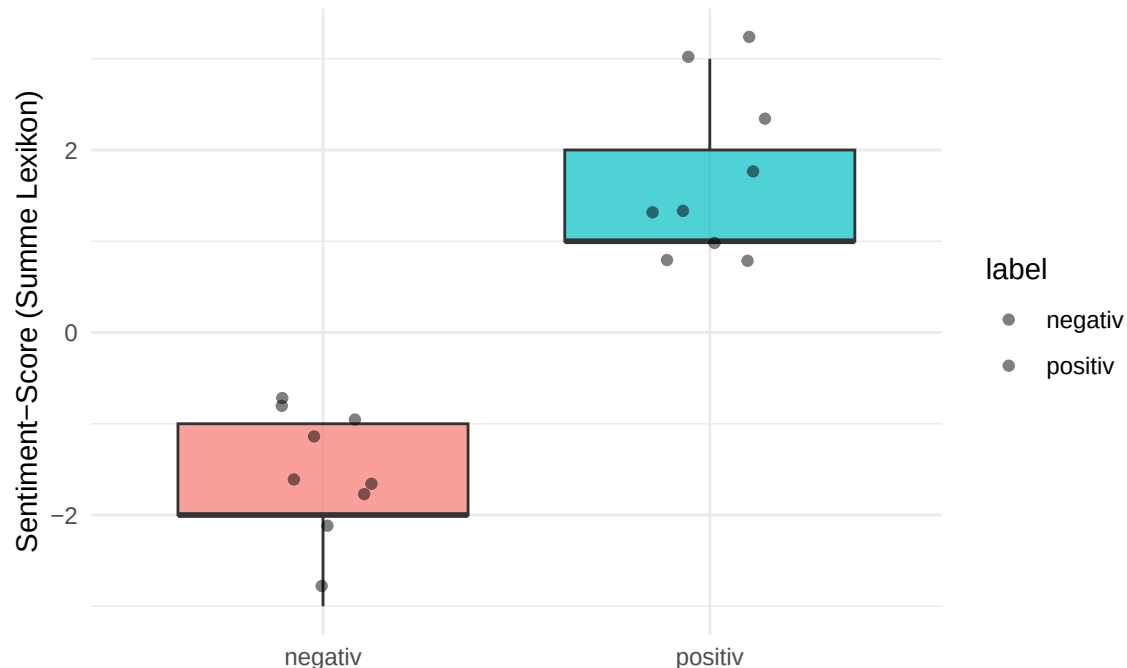
Tabelle 2: Konfusionsmatrix Test (Accuracy = 0.33)

	negativ	positiv
negativ	1	3
positiv	1	1

Hinweis: Auf dem winzigen Demo-Datensatz sind diese Zahlen NICHT aussagekräftig. Mit eurem echten Korpus (≥ 200 Texte) und ggf. Cross-Validation wird das robust. Evaluationsmasse je nach Fragestellung: Accuracy, AUC, LogLoss (vgl. W13, S.6).

7 Methode 3 – Sentimentanalyse mit Wortlisten (W12 S.22-24)

Idee: positive Wörter +, negative Wörter -, pro Text summieren. Für Englisch `get_sentiments("afinn"/"bing")`; für **Deutsch** SentiWS oder eigene Liste (W12, S.23). Eine Wortliste ist nie neutral – sie muss zu Sprache, Domäne und Fragestellung passen.



SBERT / Satz-Embeddings (W12, S.18; Python via reticulate)

```
library(reticulate)
st <- import("sentence_transformers")
model <- st$SentenceTransformer("paraphrase-multilingual-MiniLM-L12-v2")
emb <- model$encode(reviews$text) # ein dichter Vektor pro Review
# emb anschliessend als Feature-Matrix: Klassifikation, Clustering, Ähnlichkeitssuche.
```

Topic Modeling – LDA (W13)

```
library(topicmodels)
dtm <- reviews |>
  unnest_tokens(word, text, strip_numeric = T) |>
  filter(!word %in% de_stop) |>
  mutate(stem = wordStem(word, language = "de")) |>
  count(doc_id, stem) |>
  cast_dtm(doc_id, stem, n)

k <- 4 # Anzahl Topics: über Interpretierbarkeit + Coherence wählen (W13, S.19/20)
lda <- LDA(dtm, k = k, method = "Gibbs",
  control = list(seed = 42, alpha = 0.1, delta = 0.05)) # Defaults W13, S.18

tidy(lda, "beta") |> group_by(topic) |> slice_max(beta, n = 8) # Topic-Wort
tidy(lda, "gamma") # Dokument-Topic

# Kurs-Variante mit Coherence:
# library(textmineR)
# m <- FitLdaModel(dtm_matrix, k = k, ...)
# CalcProbCoherence(m$phi, dtm_matrix, M = 5)
```

LLM-Klassifikation (W12, S.19)

```
# Prompt-basiert, z.B. über eine LLM-API. Pseudostruktur:
# prompt <- "Klassifiziere das folgende Review als positiv, neutral oder negativ.
#           Antworte nur mit einem Wort. Review: {text}"
# Rolle laut Folien: Sanity Check / Labeling-Hilfe / qualitative Ergänzung.
# Beachten: Kosten, Datenschutz, Variabilität -> systematische Evaluation nötig.
```

9 Interpretation der wichtigsten Ergebnisse

Zentrale Befunde mit Bezug zur Leitfrage. Pro Befund: *Was gefunden, wie sicher, was bedeutet es im Kontext?* (W13, S.27: nicht die Modelle sind das Ziel, sondern was wir über die Texte lernen.)

10 Kritische Diskussion

Pflichtteil (W12, S.29). Hier wird bewertet.

Mindestens ansprechen (Folienbezug):

- **Grenzen BoW/TF-IDF (W12, S.5):** ignorieren Wortreihenfolge, Kontext, Mehrdeutigkeit, semantische Ähnlichkeit ("nicht schlecht" vs. "schlecht").

- **Sentiment-Wortlisten (W12, S.22):** Negation, Ironie, kontextabhängige Wörter, domänenspezifische Begriffe; Liste ist nie neutral.
- **Preprocessing (W11, S.38):** Stemming kann Bedeutung verzerren; seltene Wörter können Rauschen oder wichtige Hinweise sein.
- **Daten & Bias (W11, S.25):** Korpus-Verzerrung, Selektionsbias bei Reviews, Rechtschreibfehler (W11, S.51), Sprache/Übersetzung (W12, S.26).
- **Methodische Sauberkeit (W11, S.45):** Wortliste nur auf Trainingsdaten.
- **Topic Modeling (W13, S.21):** explorativ, kein Wahrheitsgenerator; k-Wahl und Coherence ersetzen keine inhaltliche Prüfung.

11 Fazit

Kurzer Schluss (3–5 Sätze): Antwort auf die Leitfrage + was ihr mit mehr Zeit/Daten anders machen würdet.

12 Anhang: Code

```
# --- Kern (läuft auf dem Demo-Datensatz durch) ---
library(tidyverse) # dplyr, ggplot2, stringr, tidyr
library(tidytext)  # unnest_tokens, bind_tf_idf, cast_dtm, tidy()
library(tokenizers) # tokenize_words / tokenize_ngrams (im Kurs verwendet)
library(stopwords) # stopwords(source="snowball", language="de")
library(SnowballC) # wordStem(..., language="de") -> Stemming
library(naivebayes) # naive_bayes() -> Klassifikation auf BoW (Kurs-Beispiel)
library(knitr)

# --- Optional, je nach gewählter Methode (einkommentieren) ---
# library(topicmodels) # LDA (Woche 13)
# library(textmineR)    # FitLdaModel + CalcProbCoherence (Kurs-Variante)
# library(word2vec)     # word2vec(), doc2vec() (Woche 12)
# library(reticulate)   # SBERT via Python sentence-transformers
# library(wordcloud)    # optionale Wordcloud
# =====
# TODO: Eure echten Daten laden und das Demo-Beispiel ersetzen.
# Erwartet: tibble mit doc_id | text | (optional) rating
# z.B. reviews <- read_csv("daten/reviews.csv")
# Wichtig (W11, S.51): Daten anschauen, Rechtschreibfehler korrigieren,
# Sprache prüfen (ggf. nur eine Sprache oder übersetzen, W12 S.26).
# =====

# --- DEMO-Datensatz (nur damit die Vorlage durchläuft -> ERSETZEN) ---
reviews <- tibble(
  doc_id = 1:20,
  rating = c(5,5,4,3,2,1,5,2,3,5,4,1,5,2,4,1,4,2,5,1),
  text = c(
    "Hervorragendes Essen und sehr freundlicher Service, gerne wieder.",
    "Tolle Aussicht und leckere Gerichte, absolut empfehlenswert.",
    "Schneller Service, gutes Preis-Leistungs-Verhältnis.",
    "Das Essen war okay, aber die Wartezeit war zu lang.",
    "Leider kalt serviert und unfreundliches Personal.",
    "Sehr enttäuschend, teuer und die Qualität mangelhaft.",
    "Gemütliches Ambiente, das Personal war zuvorkommend.",
    "Die Portionen waren klein für den hohen Preis.",
    "Nicht schlecht, aber auch nichts Besonderes.",
    "Fantastischer Abend, das Dessert war ein Traum.",
    "Lange Wartezeit, das Essen dann aber sehr gut.",
    "Katastrophaler Service, wir warten immer noch auf die Rechnung.",
    "Frische Zutaten und kreative Küche, top.",
    "Der Wein war überteuert und das Essen fade.",
    "Super Lage direkt am See, freundliches Team.",
    "Ganz toll, schon wieder eine halbe Stunde auf das Essen gewartet.",
    "Solides Mittagessen, nichts zu beanstanden.",
    "Das Lokal war laut und das Essen lauwarm.",
    "Aufmerksamer Service und ausgezeichnete Weinkarte.",
    "Schlechte Erfahrung, wir kämen nicht noch einmal."
  )
)
```

```

# Rating -> binäres Label (Kurs-Konvention W12, S.27: 4-5 = positiv, 1-3 = negativ)
reviews <- reviews |>
  mutate(label = factor(if_else(rating >= 4, "positiv", "negativ"),
                             levels = c("negativ", "positiv")))

reviews |>
  summarise(
    n_texte      = n(),
    anteil_positiv = mean(label == "positiv"),
    woerter_im_mittel = mean(str_count(text, "\\w+"))
  ) |>
  kable(digits = 2, caption = "Eckdaten des Korpus")
de_stop <- stopwords(source = "snowball", language = "de") # 231 Wörter (W11, S.24)

tokens <- reviews |>
  select(doc_id, label, text) |>
  unnest_tokens(word, text, strip_numeric = T) |> # Tokenisierung, Zahlen raus
  filter(!word %in% de_stop) # Stopwörter entfernen

# Stemming (W11, S.27-32). ACHTUNG: Stemming ODER Lemmatisierung ODER keins (W11, S.36).
# Stemming kann Bedeutung verzerren (S.30/31) -> bewusst entscheiden.
tokens <- tokens |>
  mutate(stem = wordStem(word, language = "de"))

# Alternative Lemmatisierung statt Stemming (W11, S.34, Paket udpipe):
# ud_model <- udpipe_download_model("german-gsd"); ...
top_stems <- tokens |> count(stem, sort = T) |> slice_head(n = 15)

ggplot(top_stems, aes(x = n, y = fct_reorder(stem, n))) +
  geom_col(fill = "steelblue") +
  labs(x = "Häufigkeit", y = NULL) +
  theme_minimal()

# Optionale Wordcloud:
# library(wordcloud)
# fr <- tokens |> count(stem, sort = T)
# wordcloud(fr$stem, fr$n, min.freq = 1, max.words = 60,
#           colors = RColorBrewer::brewer.pal(8, "Dark2"))
# Hier dient die KLASSE als "Dokument", um charakteristische Wörter zu finden.
tfidf_klasse <- tokens |>
  count(label, stem, sort = T) |>
  bind_tf_idf(stem, label, n) |>
  group_by(label) |>
  slice_max(tf_idf, n = 8, with_ties = FALSE) |>
  ungroup()

ggplot(tfidf_klasse,
  aes(tf_idf, reorder_within(stem, tf_idf, label), fill = label)) +
  geom_col(show.legend = FALSE) +
  facet_wrap(~ label, scales = "free_y") +
  scale_y_reordered() +
  labs(x = "TF-IDF", y = NULL) +
  theme_minimal()
n <- nrow(reviews)

```

```

idx <- sample(seq_len(n), size = floor(0.7 * n))
train <- reviews[idx, ]
test  <- reviews[-idx, ]
# Wortliste NUR auf Trainingsdaten (W11, S.41): Stems mit n>1 (Demo-Schwelle)
word_list <- train |>
  unnest_tokens(word, text, strip_numeric = T) |>
  filter(!word %in% de_stop) |>
  mutate(stem = wordStem(word, language = "de")) |>
  count(stem, sort = T) |>
  filter(n > 1) |>
  pull(stem)
# Baut die BoW-Feature-Matrix für ein beliebiges Subset mit FIXER word_list.
# Garantiert, dass alle word_list-Spalten existieren (wichtig fürs Test-Set).
build_features <- function(df, word_list) {
  feats <- df |>
    select(doc_id, text) |>
    unnest_tokens(word, text, strip_numeric = T) |>
    filter(!word %in% de_stop) |>
    mutate(stem = wordStem(word, language = "de")) |>
    filter(stem %in% word_list) |>
    count(doc_id, stem) |>
    pivot_wider(names_from = stem, values_from = n, values_fill = 0)

  miss <- setdiff(word_list, names(feats))  # im Subset nicht vorkommende Stems
  feats[miss] <- 0

  df |>
    select(doc_id, label) |>
    left_join(feats, by = "doc_id") |>
    mutate(across(all_of(word_list), ~ replace_na(.x, 0))) |>
    select(doc_id, label, all_of(word_list))
}

# Für Naive Bayes: nur ob ein Wort vorkommt (binär, W11 S.44)
to_binary <- function(feats_df, word_list) {
  feats_df |>
    mutate(across(all_of(word_list),
      ~ factor(.x > 0, levels = c(FALSE, T),
        labels = c("no", "yes"))))
}

train_bin <- to_binary(build_features(train, word_list), word_list) |> select(-doc_id)
test_bin  <- to_binary(build_features(test, word_list), word_list) |> select(-doc_id)

nb <- naive_bayes(label ~ ., data = train_bin, laplace = 1)
pred <- predict(nb, newdata = test_bin |> select(-label))

konfusionsmatrix <- table(Vorhersage = pred, Tatsächlich = test_bin$label)
accuracy <- mean(pred == test_bin$label)
konfusionsmatrix |> kable(caption = sprintf("Konfusionsmatrix Test (Accuracy = %.2f)", accuracy))
# SentiWS einlesen (https://wortschatz.uni-leipzig.de/de/download/SentiWS) -> ERSETZEN.
# Format je Zeile: "Wort/POS \t value \t flexionen"
# senti_lex <- read_delim(c("SentiWS_v2.0_Positive.txt", "SentiWS_v2.0_Negative.txt"),
#
delim="\t", col_names=c("wort_pos", "value", "flex")) |>

```

```

# mutate(word = str_to_lower(str_remove(wort_pos, "\\|.*"))) />
# select(word, value)

# --- Mini-Seed-Lexikon (PLATZHALTER, damit die Vorlage läuft) ---
senti_lex <- tibble(
  word = c("hervorragend", "freundlich", "tolle", "leckere", "empfehlenswert", "gut", "gerne",
    "gemütlich", "zuvorkommend", "fantastisch", "traum", "frische", "kreative", "top",
    "super", "freundliches", "aufmerksamer", "ausgezeichnete", "solides",
    "enttäuschend", "teuer", "mangelhaft", "kalt", "unfreundliches", "klein", "hohen",
    "katastrophaler", "übersteuert", "fade", "laut", "lauwarm", "schlechte", "schlecht", "lang"),
  value = c(rep(1, 19), rep(-1, 15))
)

sentiment_scores <- reviews |>
  select(doc_id, label, rating, text) |>
  unnest_tokens(word, text) |>
  inner_join(senti_lex, by = "word") |>
  summarise(sentiment = sum(value), .by = c(doc_id, label, rating))

sentiment_scores |>
  ggplot(aes(label, sentiment, fill = label)) +
  geom_boxplot(show.legend = FALSE, alpha = 0.7) +
  geom_jitter(width = 0.15, alpha = 0.5) +
  labs(x = NULL, y = "Sentiment-Score (Summe Lexikon)") +
  theme_minimal()
library(word2vec)
set.seed(42)
# Auf vorverarbeitetem Text trainieren (eine Zeichenkette pro Dokument):
w2v <- word2vec(x = train$text, type = "cbow", dim = 50,
  window = 5, iter = 20, min_count = 5)

# ähnliche Wörter explorieren:
predict(w2v, c("service", "preis", "essen"), type = "nearest", top_n = 5)

# Dokumentvektor = Mittelwert der Wortvektoren (W12, S.14) -> Feature für ML:
doc_vec <- doc2vec(w2v, newdata = reviews$text)
# vortrainiertes Modell laden statt selbst trainieren:
# model_de <- read.word2vec("model.bin", normalize = T)
library(reticulate)
st <- import("sentence_transformers")
model <- st$SentenceTransformer("paraphrase-multilingual-MiniLM-L12-v2")
emb <- model$encode(reviews$text) # ein dichter Vektor pro Review
# emb anschliessend als Feature-Matrix: Klassifikation, Clustering, Ähnlichkeitssuche.
library(topicmodels)
dtm <- reviews |>
  unnest_tokens(word, text, strip_numeric = T) |>
  filter(!word %in% de_stop) |>
  mutate(stem = wordStem(word, language = "de")) |>
  count(doc_id, stem) |>
  cast_dtm(doc_id, stem, n)

k <- 4 # Anzahl Topics: über Interpretierbarkeit + Coherence wählen (W13, S.19/20)
lda <- LDA(dtm, k = k, method = "Gibbs",
  control = list(seed = 42, alpha = 0.1, delta = 0.05)) # Defaults W13, S.18

```

```

tidy(lda, "beta") |> group_by(topic) |> slice_max(beta, n = 8)  # Topic-Wort
tidy(lda, "gamma")                                           # Dokument-Topic

# Kurs-Variante mit Coherence:
# library(textmineR)
# m <- FitLdaModel(dtm_matrix, k = k, ...)
# CalcProbCoherence(m$phi, dtm_matrix, M = 5)
# Prompt-basiert, z.B. über eine LLM-API. Pseudostruktur:
# prompt <- "Klassifiziere das folgende Review als positiv, neutral oder negativ.
#           Antworte nur mit einem Wort. Review: {text}"
# Rolle laut Folien: Sanity Check / Labeling-Hilfe / qualitative Ergänzung.
# Beachten: Kosten, Datenschutz, Variabilität -> systematische Evaluation nötig.

```